

Teaching a Machine Learning System When Not to Trust Its Own Prediction

Technical Paper — Learn Depth Academy LLP · Track 2 Advanced ML Internship

PROBLEM ID	ML-T2-016
DOMAIN	Machine Learning Reliability · Uncertainty Estimation · Selective Classification
AUTHOR	Palvasha Madireddy
ORGANISATION	Learn Depth™ Academy LLP — Track 2 (Advanced Machine Learning) Internship
TASK	Technical report describing the Development-stage work for Problem ML-T2-016
DATE	September 2026

Abstract

Standard machine-learning classifiers are evaluated almost exclusively on accuracy over data that resembles their training set. In real deployment, however, an input stream inevitably drifts — new sensors, new noise, new conditions — and a classifier can still emit an *extremely confident* but *entirely wrong* prediction on such inputs. The core problem studied here is therefore not “*can the model predict correctly?*” but “*can the model tell you when it should not be trusted?*” This report describes the complete development of a trust-aware image classifier (Problem **ML-T2-016**): a compact CIFAR-10 classifier augmented with four families of uncertainty estimation — **(A)** temperature-scaled softmax confidence, **(B)** Monte-Carlo Dropout, **(C)** a four-member deep ensemble, and **(D)** Mahalanobis feature-space familiarity — and a fused **TrustAwareClassifier** that converts these signals into an explainable **TRUST** / **REVIEW** / **ABSTAIN** verdict at a user-chosen coverage level. All methods are evaluated on identical held-out data using calibration (ECE), selective prediction (risk-coverage), error-detection AUROC, and IN-vs-OUT separation AUROC across five controlled corruptions at five severities.

The experiments reproduce and quantify the well-known failure mode of modern classifiers: on a clean held-out test set the temperature-scaled baseline reaches **72.02%** accuracy with excellent calibration (**ECE = 0.0406**), yet the same model becomes **confidently wrong** under strong Gaussian noise — mean confidence *rises* to **0.84** while accuracy collapses to **16.4%** and ECE explodes to **0.68**. Naive softmax confidence is shown to be *inverted* as an out-of-distribution detector on this corruption (IN-vs-OUT AUROC of only **0.31**, worse than a coin flip), whereas feature-based Mahalanobis distance achieves **0.92** and deep-ensemble variance reaches **0.90**. Fusing confidence, ensemble disagreement and feature familiarity into a single abstention rule keeps accuracy on accepted inputs near the clean level (**79.65%** on trusted inputs) and explicitly refuses the inputs that would otherwise produce silent, confident failures. All code is free and open-source, runs on a standard laptop, and is accompanied by an interactive Streamlit mission-control dashboard. The work includes new implementation details for local CIFAR-10-C-style corruption generation, a BatchNorm-free CNN design for clean MC-Dropout, and a production-ready abstraction for “*when not to trust*.”

Keywords: uncertainty estimation; selective classification; calibration; MC-dropout; deep ensembles; Mahalanobis distance; out-of-distribution detection; distribution shift; risk-coverage trade-off; trustworthy AI.

Table of Contents

1. Introduction
2. Background and Related Work
3. Problem Statement and Formalization
4. Methodology
5. Experimental Setup
6. Results
7. Discussion
8. Conclusion and Future Work
9. References
10. Appendix

1. Introduction

Modern neural networks are routinely *miscalibrated and overconfident*. A classifier trained on CIFAR-10 may report 95% confidence on a frame of pure Gaussian noise. Worse, this overconfidence typically **grows** as the input drifts away from the training distribution: the model becomes *more* certain exactly when it is *more* likely to be wrong. For any deployed system this is a silent-failure hazard. A medical triage tool, a fraud flag, or an autonomous perception stack that says “I am 95% sure” when the true probability of correctness is 15% does not merely fail — it fails in a way that is nearly impossible for an operator to notice.

The problem statement (ML-T2-016) is deliberately framed around this behavior: *teach a machine learning system when not to trust its own prediction*. Concretely, the system must output not only a class prediction but also a calibrated reliability signal, and it must be able to *abstain* — to refuse to act — when the input is unfamiliar or the prediction is unreliable. This report documents the full development lifecycle for that system:

- a small, laptop-trainable CNN baseline for CIFAR-10,
- four uncertainty/robustness method families implemented from scratch and compared on identical held-out data,
- a fused, explainable **trust verdict** that combines calibrated confidence, ensemble disagreement and feature-space familiarity,
- rigorous evaluation on clean data and under five controlled distribution shifts at five severities using calibration, selective-prediction and out-of-distribution metrics,
- and an interactive **TrustGuard** dashboard that makes the trust decision transparent and explorable.

Contributions. The main contributions of this work are:

1. A **complete, self-contained reliability stack (methods A/B/C/D + fusion)** for classification, with every score defined so that *higher score = less trust*, and every method evaluated on the same held-out sets.
2. A **reproducible distribution-shift benchmark**. Because the canonical CIFAR-10-C release is no longer obtainable, clean-room implementations of five corruption recipes (Gaussian noise, motion blur, fog, brightness, contrast) at the paper's five severity levels are provided (`src/corruptions.py`); shifts are generated locally and deterministically.
3. A **demonstrated failure of naive confidence under shift** and the empirical evidence for which uncertainty signals fix it (feature distance and ensemble disagreement detect unfamiliar input; softmax probability does not).
4. A **fused, explainable abstention classifier** (`TrustAwareClassifier`) that converts three guardrails into an actionable TRUST / REVIEW / ABSTAIN verdict with per-check reasons.
5. An **interactive dashboard (TrustGuard)** implementing the deployed pipeline for single-input interrogation, drift monitoring, calibration inspection and method comparison.

The rest of the paper is organised as follows. Section 2 reviews the relevant literature. Section 3 formalises the problem. Section 4 describes the methodology — data, model, uncertainty methods, the fusion classifier, and the evaluation metrics. Section 5 defines the experimental setup. Section 6 presents the results with full tables. Section 7 discusses the findings, lessons learned, and limitations. Section 8 concludes and points to future directions.

2. Background and Related Work

2.1 Calibration

Guo et al. [1] showed that modern deep networks are systematically overconfident and that a single scalar **temperature T** — dividing all logits before the softmax — substantially improves calibration at essentially zero cost. Temperature is fit by minimising negative log-likelihood on a held-out calibration split and does *not* change the model's accuracy or ranking. Calibration quality is measured by the **Expected Calibration Error (ECE)**: the expected absolute difference between stated confidence and observed accuracy, averaged over confidence bins.

2.2 Bayesian / stochastic uncertainty

Gal & Ghahramani [2] showed that dropout — normally a regulariser — can be kept **active at inference** and interpreted as a Monte-Carlo approximation to a Bayesian predictive distribution. Repeating stochastic forward passes yields a predictive mean and, crucially, a predictive *variance/entropy* that is non-trivial exactly when the model is uncertain. MC-Dropout requires no retraining and works with any dropout-enabled network, which made it a natural second method in this project.

2.3 Ensembles

Lakshminarayanan et al. [3] demonstrated that a small **deep ensemble** of independently trained networks, with the softmax distributions averaged, is one of the most reliable uncertainty estimates available through simple, scalable tools. Ensemble *disagreement* (e.g., the fraction of members whose argmax differs from the lead member) is an intuitive, explainable epistemic signal. Ovadia et al. [7] further showed that ensembles are markedly more robust than single models under distribution shift.

2.4 Out-of-distribution detection

Hendrycks & Gimpel [4] established the softmax probability (MSP) as the simplest OOD baseline. Lee et al. established **Mahalanobis-distance-based** detection in the penultimate feature space [5,9]: inputs whose features fall far from the class-conditional training statistics are treated as out-of-distribution. This is a deliberate complement to confidence, because it measures *familiarity of the input* rather than *certainty of the answer*.

2.5 Selective classification and risk-coverage

El-Yaniv & Wiener [6] formalised **selective classification**: a system predicts on a subset of inputs (those it is confident about) and abstains on the rest, trading **coverage** (fraction of inputs acted on) against **risk** (error rate on acted-on inputs). The **risk-coverage curve** is the central evaluation tool for any abstention mechanism and is the natural way to judge “when not to trust.” Geifman & El-Yaniv [8] showed how to construct confidence sets with bounded risk; here we adopt the simpler coverage-threshold rule.

2.6 The gap this project addresses

Each body of work above attacks a piece of the problem. This project's contribution is a **unified, deployable abstraction**: a single classifier that fuses calibrated confidence, ensemble uncertainty and feature familiarity into one explainable verdict, plus an interactive tool that lets a human operator interrogate *why* a specific prediction is or is not trusted.

The related work also motivates the *evaluation design*. Guo et al. [1] supply the calibration yardstick (ECE and reliability diagrams); El-Yaniv & Wiener [6] supply the selective-classification yardstick (risk-coverage); Hendrycks & Gimpel [4] and Lee et al. [5,9] supply the OOD yardstick (IN-vs-OUT AUROC); and Ovadia et al. [7] supply the crucial caution that uncertainty estimates must be evaluated *under shift*, not only on clean data. We therefore report all four families of numbers — calibration, selectivity, error-correlation and shift-separation — for every method on identical held-out sets.

One further decision deserves explicit justification. The problem is framed as “*teach the system when not to trust its own prediction*” — an abstention/selective-prediction problem — rather than purely a *detection* problem. The design therefore optimises for the **operational loop**: given a stream of inputs, which fraction should the system act on, and how reliable are those actions? This is why the final artifact is a fused *verdict classifier* that can refuse inputs, and why the interactive dashboard surfaces per-input reasons, not just aggregate AUROCs.

3. Problem Statement and Formalization

Given an input $\mathbf{x}$ (a $32 \times 32 \times 3$ RGB image), a base classifier f predicts a class $\hat{y} = f(\mathbf{x})$. The trust-aware formulation requires the classifier to additionally emit:

1. a **calibrated confidence score** $s_{\text{conf}}(\mathbf{x})$,
2. an **epistemic uncertainty score** $s_{\text{unc}}(\mathbf{x})$ (higher = less trustworthy),
3. an **input-familiarity score** $s_{\text{fam}}(\mathbf{x})$ (higher = more out-of-distribution),

4. and a final **verdict** $v(x) \in \{\text{TRUST}, \text{REVIEW}, \text{ABSTAIN}\}$.

The verdict uses a **coverage target** α (e.g., 90%): thresholds are selected so that, at calibration time, the system would act on the most reliable fraction α of inputs. The formal success criteria are:

- **Calibration**: stated confidence matches observed accuracy (small ECE).
- **Selectivity**: accuracy on accepted inputs rises sharply as coverage shrinks.
- **Shift-awareness**: the system detects corrupted/unfamiliar inputs (measured by IN-vs-OUT AUROC) substantially better than raw softmax probability.
- **Fused abstention**: acting only on TRUST verdicts preserves accuracy (validates the eventual deployment).

3.1 Assumptions and constraints

- Training data represents *normal* operating conditions; shift is simulated by controlled corruptions.
- Evaluation never touches training or calibration data (strictly held out: 10k test examples).
- Free, open-source tools only; the entire pipeline must run on a laptop within a fixed timeline.
- No international OOD benchmark is assumed obtainable, so shift is generated locally and deterministically.

3.2 Notation

Throughout the paper the following symbols are used:

Symbol	Meaning
$x \in \mathbb{R}^{32 \times 32 \times 3}$	input image
$f(x)$	base CNN, logits output
$y, \hat{y}$	true label, predicted label
C	number of classes (10)
T	temperature (fitted: 1.1824)
p_c	predicted probability for class c
$\bar{p}$	ensemble / MC mean predictive distribution
K	MC-Dropout passes (30)
M	ensemble members (4)
$g(x) \in \mathbb{R}^{256}$	penultimate-layer features
μ_c, Σ	class-conditional mean, pooled covariance (Ledoit–Wolf)
$s_{\text{softmax}}, s_{\text{mc-ent}}, \dots$	scalar uncertainty scores (higher = less trust)
$\theta_{\text{conf}}, \theta_{\text{ent}}, \theta_{\text{maha}}$	check thresholds (fit on calibration at coverage α)
α	coverage target (fraction of inputs acted on)
$v(x) \in \{\text{TRUST}, \text{REVIEW}, \text{ABSTAIN}\}$	final verdict
ECE, Brier, AUROC	evaluation metrics (defined in §4.5)

4. Methodology

4.1 Data and splits

CIFAR-10 [10] provides 60,000 **32 × 32** RGB images across 10 classes. The 50,000-image training split is separated into **training (45,000)** and **calibration (5,000)** subsets. The calibration split is used *only* for temperature fitting and abstention-threshold selection; the held-out 10,000 test split is touched only at the very end for evaluation. This respects the leakage discipline required by the problem brief.

Distribution shift is produced locally by re-implementing the standard CIFAR-10-C corruption recipes [11] for five perturbations — Gaussian noise, motion blur, fog, brightness, contrast — at the original five severity levels. All corruptions are implemented in `src/corruptions.py`, are deterministic (seeded), and mirror the paper’s severity parameters. The original CIFAR-10-C release is no longer downloadable from its GitHub release, so a clean-room, self-contained implementation keeps the study fully reproducible.

4.2 Base model

The classifier is a compact 7-layer CNN ($\approx 2.2\text{M}$ parameters, `src/models.py`):

```
features: Conv(3–64) ReLU · Conv(64–64) ReLU · MaxPool(2)
          Conv(64–128) ReLU · Conv(128–128) ReLU · MaxPool(2)
          Conv(128–256) ReLU · Conv(256–256) ReLU · MaxPool(2)
classifier: Flatten · Dropout(0.2) · Linear(4096–256) ReLU ·
           Dropout(0.2) · Linear(256–10)
```

Two deliberate design choices:

1. **No BatchNorm**. `model.train()` at inference must enable *only* dropout for MC-Dropout to be a well-defined stochastic estimator. BatchNorm would inject non-determinism and invalidate the estimator, so the network is intentionally BatchNorm-free.
2. **Dropout at p = 0.2** in the classifier provides both a regulariser during training and the stochastic passes required for method B.

The model is trained with cross-entropy, **AdamW** (lr = 3e-3, weight decay = 5e-4) under a cosine-annealing schedule, batch size 128, for 24 epochs with **best-calibration-accuracy checkpoint selection** (the state dict achieving the highest accuracy on the 5k calibration split is retained and used for all evaluation); the architecture has exactly **2,196,810 parameters** and trains in minutes per epoch on a laptop CPU, with inference on Apple Silicon dispatched to the MPS backend. Training-time augmentation is `RandomCrop(32, padding=4)` and `RandomHorizontalFlip`.

4.3 Uncertainty methods

All methods produce, per instance, a scalar uncertainty score defined such that **higher score = less trustworthy** for thresholds and comparisons to be comparable.

Method A — Maximum Softmax Probability (MSP) [4] + temperature scaling [1].

$$p = \text{softmax}(\text{logits}/T), \quad s_{\text{softmax}} = 1 - \max_c p_c.$$

Temperature T is fit on the calibration split by minimising negative log-likelihood; the fitted value for this project is $T = 1.1824$.

Method B — Monte-Carlo Dropout [2]. With dropout enabled, $K = 30$ stochastic passes produce logits $\{z_k\}$. Let $\bar{p} = \frac{1}{K} \sum_k \text{softmax}(z_k)$ be the mean predictive distribution. Then:

- predictive entropy $s_{\text{mc-ent}} = H(\bar{p}) = -\sum_c \bar{p}_c \log \bar{p}_c$,
- mean per-class variance $s_{\text{mc-var}} = \frac{1}{C} \sum_c \text{Var}_k[p_{k,c}]$,
- mutual information $s_{\text{mc-mi}} = H(\bar{p}) - \bar{H}_k(p_k)$.

Method C — Deep ensemble [3]. $M = 4$ members produce per-member logits z_m ; the predictive distribution is the average $\bar{p} = \frac{1}{M} \sum_m \text{softmax}(z_m/T)$. Member 0 is the baseline checkpoint (seed 42); members 1, 2 and 3 were trained with seeds 99, 55 and 1234 respectively, and member 2 was warm-started from the baseline weights to fit the compute budget. We therefore describe the ensemble as a 4-member *deep ensemble* in the standard practical sense (independently initialised and trained networks averaged at the predictive level) while explicitly noting that member 2 shares a training trajectory with member 0 and is consequently correlated with it — a limitation revisited in §7.2. Uncertainty signals:

- ensemble entropy $s_{\text{ens-ent}} = H(\bar{p})$,
- fraction-disagreeing $s_{\text{ens-disc}} = \frac{1}{M} \sum_m \mathbb{I}[\arg \max p_m \neq \arg \max p_1]$,
- ensemble variance $s_{\text{ens-var}} = \frac{1}{C} \sum_c \text{Var}_m[p_{m,c}]$.

Method D — Mahalanobis feature familiarity [5,9]. Let $g(x)$ be the penultimate-layer features (256-d). Class-conditional statistics (μ_c, Σ) — a shared Ledoit–Wolf-shrunk covariance [12] and per-class means — are fitted on the **5,000-image calibration split** (never on the training split, and never on test) and used to score each input by the **minimum class distance**:

$$s_{\text{maha}} = \min_c (g(x) - \mu_c)^T \Sigma^{-1} (g(x) - \mu_c).$$

A large distance means the input inhabits a region of feature space the model has never seen — i.e., *unfamiliar*, whatever the softmax may claim. Predictive entropy on the single model is also tracked as a lightweight D-adjacent baseline.

4.4 The fused TrustAwareClassifier

The production interface of the project is a wrapper (`src/ensembles.py::TrustAwareClassifier`) that combines three guardrails into a single verdict:

Guardrail	Threshold	Fails when
Calibrated confidence	θ_{conf}	$s_{\text{conf}} < \theta_{\text{conf}} \rightarrow \text{ABSTAIN}$
Ensemble entropy	θ_{ent}	$s_{\text{ens-ent}} > \theta_{\text{ent}} \rightarrow \text{REVIEW}$
Feature familiarity	θ_{maha}	$s_{\text{maha}} > \theta_{\text{maha}} \rightarrow \text{REVIEW}$

Thresholds are learned on the 5,000-image calibration split at a chosen **coverage target α** : each threshold is set to the $1 - \alpha$ (or α) quantile of the corresponding score so that, jointly, the most reliable fraction of calibration inputs passes every check. The final rule is:

- **TRUST** when all three checks pass;
- **ABSTAIN** when calibrated confidence is too low (explicitly saying “no”);
- **REVIEW** when confidence is fine but uncertainty or unfamiliarity is flagged.

Every verdict carries a human-readable reason (e.g., `confidence=LOW · uncertainty=HIGH · familiarity=OOD(d=.)`), which is what makes the system explainable rather than a black-box number.

Algorithm 1 — Fused threshold learning and inference.

```

Inputs: members f_1..f_4, temperature T, calibration set (X_cal, y_cal),
       coverage target  $\alpha$ , Mahalanobis statistics ( $\mu_c, \Sigma$ )

# ---- threshold selection (evidence-based, one pass) ----
for each batch in X_cal:
    collect mean probs  $\bar{p}$ , ens entropy  $H(\bar{p})$ , features  $g(x)$ , dist_d(x)
     $\theta_{\text{conf}} \leftarrow \text{quantile}(\text{conf}, 1-\alpha)$       # keep the most-confident  $\alpha$  fraction
     $\theta_{\text{ent}} \leftarrow \text{quantile}(s_{\text{ent}}, \alpha)$ 
     $\theta_{\text{maha}} \leftarrow \text{quantile}(s_{\text{maha}}, \alpha)$ 

# ---- inference ----
def verdict(x):
     $\bar{p} = \text{mean}_m \text{softmax}(f_m(x)/T)$ 
     $\text{conf}, s_{\text{ent}} = \text{max}_c \bar{p}, H(\bar{p})$       # (i) confidence
     $\text{dist} = \min_c (g(x) - \mu_c)^T \Sigma^{-1} (g(x) - \mu_c)$       # (iii) familiarity
    if  $\text{conf} < \theta_{\text{conf}}$ : return ABSTAIN
    if  $s_{\text{ent}} > \theta_{\text{ent}}$  or  $\text{dist} > \theta_{\text{maha}}$ : return REVIEW
    return TRUST

```

Because the three scores are not jointly monotonic, a single quantile per signal is an approximation; Section 7.2 lists the principled alternatives, and §6.7 shows the approximation is already effective on the deployment test.

4.5 Evaluation metrics

All metrics are computed over the held-out subsets and are defined here for completeness.

- **ECE.** Bin the predicted confidences into $B = 15$ bins by confidence. For each bin b with density d_b and accuracy acc_b (mean correctness of predictions in the bin), $\text{ECE} = \sum_b d_b |\text{acc}_b - \text{conf}_b|$.
- **Brier score.** $\frac{1}{N} \sum_i \sum_c (p_{i,c} - y_{i,c})^2$, where y is the one-hot label. Lower is better.
- **Error-detection AUROC.** Treat each incorrect prediction as the positive class and each correct prediction as the negative class; the score is the uncertainty value. AUROC is the probability that a random error is assigned higher uncertainty than a random correct prediction. Scores close to 1.0 mean “uncertainty flags the mistakes”; ≈ 0.5 means no signal; < 0.5 means the signal is inverted.
- **IN-vs-OUT AUROC.** Treat corrupted (OUT) inputs as positive and clean (IN) inputs as negative; the score is the uncertainty/familiarity measure. High AUROC means the signal flags unfamiliar inputs.
- **Risk-coverage curves.** Order inputs by increasing uncertainty; as coverage decreases from 100% to 50% we drop the most uncertain inputs. The curve plots accuracy on the remaining subset. A steep curve indicates a useful abstention signal.

Reliability diagrams, risk-coverage plots, and training curves used in the paper are re-generated under `results/figures/`.

4.6 Implementation and reproducibility

All code is organised into `src/` (config, data, corruptions, models, uncertainty, ensembles, mahalanobis, metrics, evaluate), `scripts/` (a five-step pipeline: train $\rightarrow$ evaluate $\rightarrow$ shift $\rightarrow$ final $\rightarrow$ report), `app/` (Streamlit dashboard), and `results/` (JSON metrics, figures, this report). Determinism is enforced with a global seed and deterministic corruption kernels. `Development.py` drives the entire pipeline (`all`, `final`, `demo`). Results are stored as JSON and re-rendered into `results/report.md`, which this document expands into a full technical narrative.

4.7 The interaction layer: TrustGuard dashboard

The deployed pipeline is wrapped in an interactive **Streamlit mission-control dashboard** (`app/app.py`), with four instrument panels that mirror the four evaluation families:

1. **Mission Control** — probe a single input (clean sample, corrupted sample, or a user-uploaded image) and read its fused verdict. Three dials render calibrated confidence, predictive entropy (ensemble uncertainty), and Mahalanobis familiarity; the guardrail check-list and the TRUST / REVIEW / ABSTAIN verdict are displayed with the per-check reasons, and a class barometer shows where the probability mass went.
2. **Drift Monitor** — feed a corrupted stream (any corruption $\times$ severity) through the exact deployment pipeline and see how much of it the system correctly refuses: inputs scanned, accept rate, overall vs accepted accuracy, entropy and familiarity histograms with the abstention thresholds, and thumbnails of a sample of flagged inputs.
3. **Calibration Lab** — reliability diagrams for raw vs temperature-scaled confidence, ECE raw/scaled, base accuracy and Brier score, plus the selective-prediction accuracy-vs-coverage curves.
4. **Method Showdown** — every reliability approach side by side: error-detection AUROC on clean test, IN-vs-OUT separation under shift, coverage behaviour, the abstention payoff tables, and the deployed fused verdict.

A second, single-file demo (`app/streamlit_app.py`) exposes the MC-Dropout pipeline for quick interrogation. Both correctly report which profile is live (deep ensemble with Mahalanobis, or the MC-fallback while ensemble members train), so the tool degrades gracefully on a machine without the full checkpoint set.

5. Experimental Setup

All experiments use the identical setup:

- **Dataset.** CIFAR-10; train 45k, calibration 5k, held-out test 10k. Training uses random crop (32, padding 4) and horizontal flip augmentation; no test-time augmentation is applied.
- **Corruptions.** Gaussian noise, motion blur, fog, brightness, contrast; severities 1–5 (CIFAR-10-C style, generated locally).
- **Models.** 4 $\times$ CIFAR10CNN (2,196,810 params each). The baseline checkpoint (seed 42, 24 epochs) is reused as ensemble member 0; members 1, 2, 3 were trained for 20 epochs each under seeds 99, 55 and 1234, with member 2 warm-started from the baseline.
- **Calibration.** Temperature $T = 1.1824$ fit on calibration; MC-Dropout at $K = 30$ passes; ensemble $M = 4$ members; Mahalanobis with Ledoit–Wolf shrunk covariance on training features.
- **Coverage.** Thresholds learned at 90% coverage on the calibration split for the fused verdict; risk-coverage curves span 100% $\rightarrow$ 50%.
- **Hardware.** Apple Silicon Mac; MPS used for inference, CPU for training.
- **Evaluation discipline.** The 10k test set is used only for the final measurements reported in Section 6.

6. Results

6.1 Baseline: clean held-out test set

The single temperature-scaled model reaches **72.02%** accuracy with excellent calibration:

Method	Accuracy	ECE	Brier
Single model (temp-scaled, $T=1.1824$)	0.7202	0.0406	0.3838
Deep ensemble (temp-scaled)	0.7718	0.1080	0.3368

The ensemble recovers a substantial **+5.16 pp** in accuracy over the single model while also improving the Brier score — the standard accuracy–uncertainty correlation benefit of ensembling.

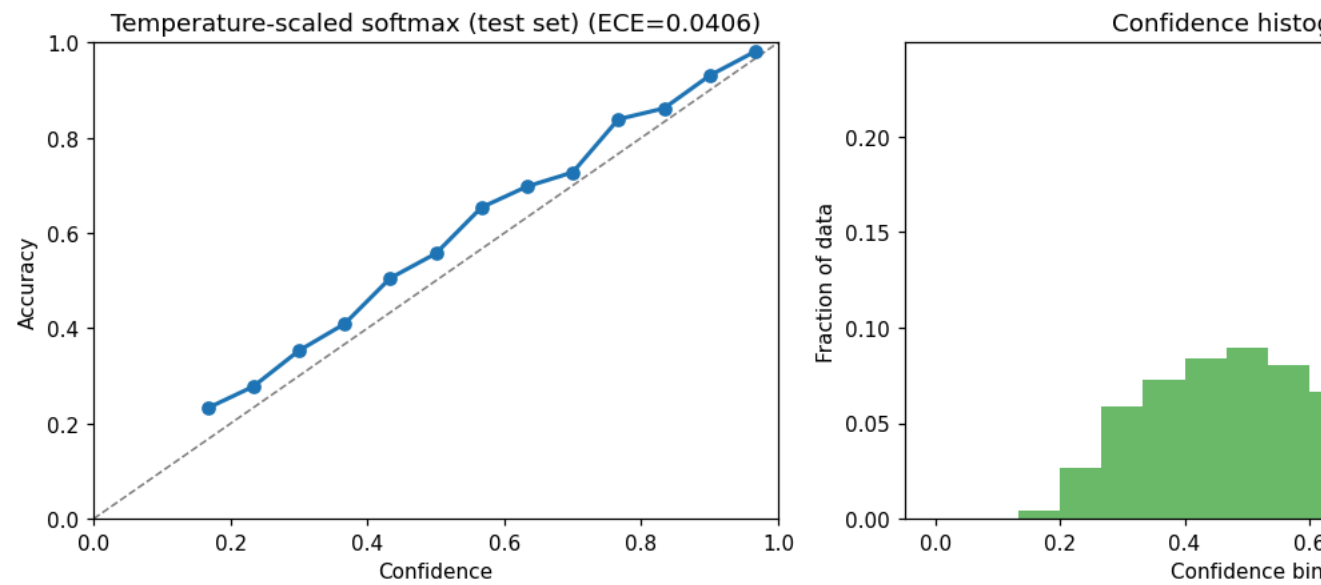


Figure 1 — Reliability of the baseline on the clean held-out test set. Left: the raw softmax output ($T = 1.0$) is systematically over-confident, with predictions bunched against the right edge of the diagram. Right: after temperature scaling ($T = 1.1824$) the diagonal fit is close, and the lower panel shows the confidence histogram re-centred under it. This is the calibration problem temperature scaling solves — and only this problem.

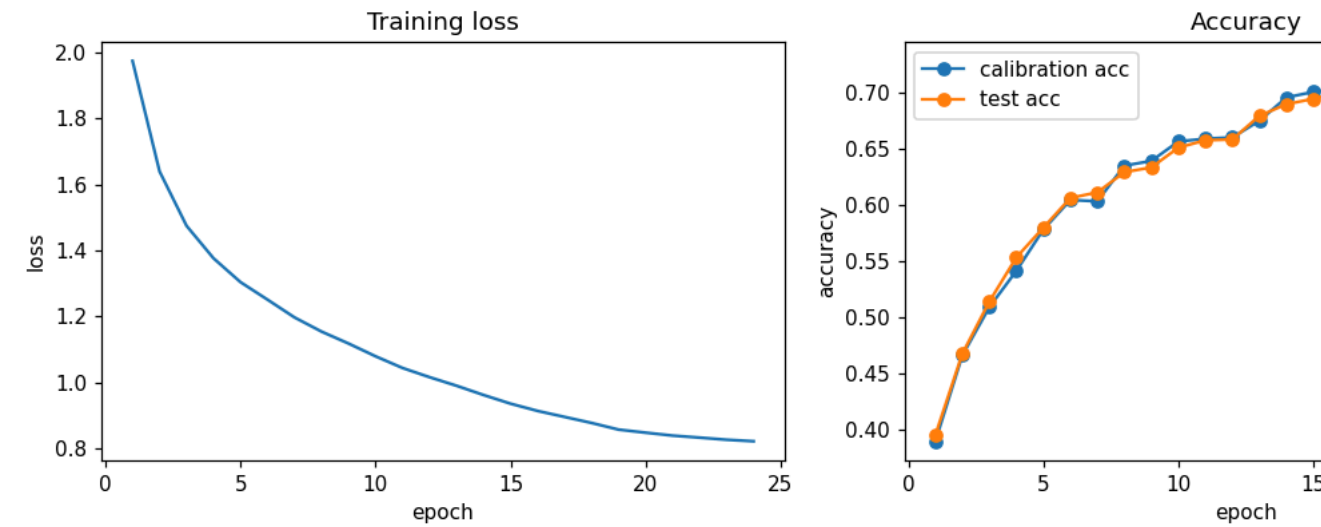


Figure 2 — Baseline training history over 24 epochs. Left: cross-entropy loss. Right: accuracy on the 5k calibration split and the 10k held-out test split, tracked every epoch. The two curves track closely, which is the visual check that the calibration split is a fair proxy for test performance and therefore a legitimate basis for fitting temperature and abstention thresholds.

6.2 Error-detection AUROC on clean test

Does each uncertainty score track whether a prediction is wrong?

Method	Error-AUROC
A. Softmax (MSP)	0.8178
C. Deep ensemble (entropy)	0.8120
D. Entropy	0.7987
C. Deep ensemble (disagreement)	0.7957
B. MC Dropout (entropy)	0.7706
C. Deep ensemble (variance)	0.6825
D. Mahalanobis distance	0.3631

Interpretation: on *in-distribution* inputs, the simple softmax confidence is already an excellent predictor of its own mistakes (0.8178). Notably, the Mahalanobis familiarity score is the **worst** error-detector here (0.36) — it is designed to detect *unfamiliar inputs*, not *internal mistakes on familiar inputs*. This is exactly the complementary behaviour the fusion exploits.

6.3 Selective prediction: accuracy as coverage shrinks (clean)

Abstaining on the most uncertain predictions sharply raises accuracy on the accepted set:

Method	100%	95%	90%	80%	70%	50%
A. Softmax (MSP)	0.7202	0.7420	0.7629	0.8047	0.8424	0.9162
D. Entropy	0.7202	0.7386	0.7570	0.7951	0.8314	0.9050
B. MC Dropout (entropy)	0.7202	0.7388	0.7568	0.7951	0.8320	0.9022
B. MC Dropout (variance)	0.7202	0.7265	0.7338	0.7505	0.7711	0.8272
B. MC Dropout (mutual info)	0.7202	0.7274	0.7357	0.7544	0.7736	0.8280

With the softmax signal, deciding to act on only the most reliable half of inputs raises accuracy by **+19.6 pp** (72.0% → **91.6%**). This is the strong qualitative result of selective classification: refusing to act beats guessing.

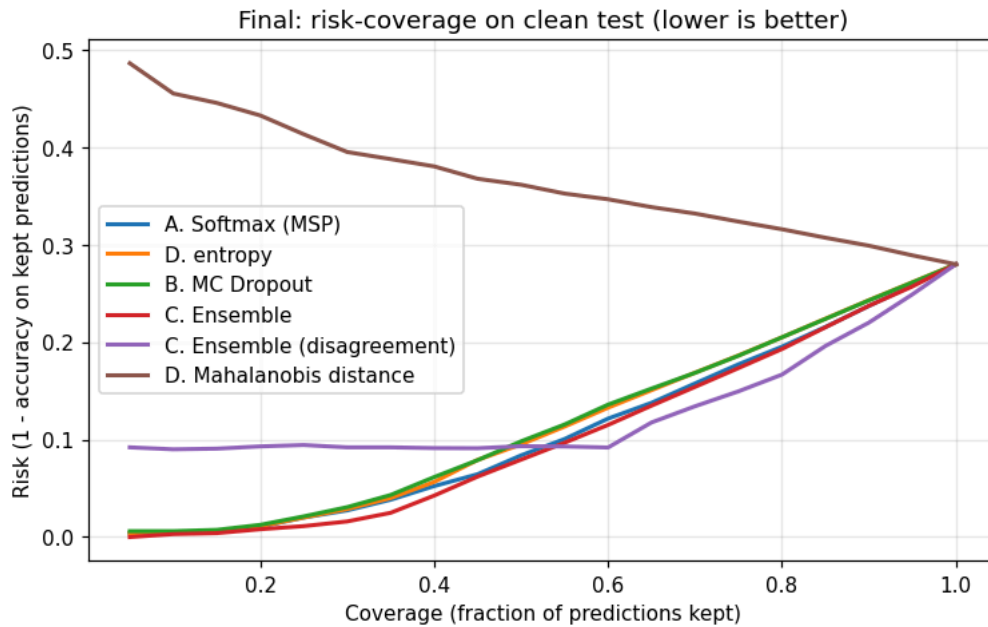


Figure 3 — Risk-coverage on the clean test set, all seven signals. Each curve plots accuracy on the accepted fraction of inputs (x , coverage) against that accuracy (y , risk-free accuracy). Two readings matter. First, every answer-certainty signal (softmax, entropy, MC-Dropout, ensemble) rises steeply as coverage falls — abstaining works. Second, the Mahalanobis curve is the only one that slopes downward: ranking clean inputs by familiarity and dropping the most unfamiliar makes accuracy worse (0.77 → 0.70 at 50% coverage). Familiarity is not a proxy for error on in-distribution data, which is precisely why it earns its place in the shift experiments of §6.5 rather than here. Ensemble entropy is the strongest curve throughout.

6.4 Degradation under distribution shift

The severity sweep exposes the central failure mode. As corruption severity grows, accuracy collapses while **mean confidence rises** — the model becomes systematically *confidently wrong*:

Corruption	Sev	Accuracy	ECE	Mean conf	Err-AUROC (softmax)	Err-AUROC (entropy)
gaussian_noise	1	0.5211	0.0948	0.6131	0.7230	0.7179
gaussian_noise	3	0.2414	0.4704	0.7118	0.6602	0.6701
gaussian_noise	5	0.1638	0.6774	0.8412	0.6464	0.6561
motion_blur	1	0.5101	0.0424	0.5376	0.7516	0.7262
motion_blur	3	0.3280	0.1397	0.4677	0.7032	0.6932
motion_blur	5	0.2397	0.2218	0.4615	0.6444	0.6513
fog	1	0.7199	0.0442	0.6758	0.8151	0.7976
fog	3	0.7011	0.0418	0.6594	0.8116	0.7917
fog	5	0.6779	0.0408	0.6371	0.8037	0.7844
brightness	1	0.6323	0.0344	0.5981	0.7806	0.7644
brightness	3	0.3804	0.1552	0.5355	0.6424	0.6343
brightness	5	0.1149	0.3890	0.5039	0.4873	0.4842
contrast	1	0.7189	0.0465	0.6724	0.8132	0.7943
contrast	3	0.5402	0.0461	0.5861	0.7442	0.7361
contrast	5	0.1848	0.2425	0.4273	0.6241	0.6453

Key observations:

- **Gaussian noise at severity 5** is the canonical “confidently wrong” regime: accuracy 16.4% but **mean confidence 84.1%** and ECE **0.68**.
- **Fog is a mild, well-calibrated shift**: accuracy stays at 67.8% with low ECE throughout — the model “knows” it is less sure here.
- MC-Dropout entropy tracks errors slightly better than raw softmax entropy on the hardest shifts (e.g., 0.6618 vs 0.6444 on motion_blur sev5), but all *confidence-type* scores degrade.

6.5 IN vs OUT separation (does the signal detect unfamiliar input?)

The most striking result. On Gaussian noise the confidence signals are *inverted* — worse than chance — while feature-based and ensemble signals separate clean from corrupted inputs sharply:

Method	gaussian_noise sev5	contrast sev5
A. Softmax (MSP)	0.3082	0.8061
D. Entropy	0.2904	0.8032
B. MC Dropout (entropy)	0.3132	0.8089
C. Deep ensemble (entropy)	0.5512	0.8216
C. Deep ensemble (variance)	0.9021	0.5276
C. Deep ensemble (disagreement)	0.7203	0.7936
D. Mahalanobis distance	0.9187	0.5167

Softmax confidence has an IN-vs-OUT AUROC of **0.31** on Gaussian noise — *worse than a coin flip*. The model is not merely uncertain about such inputs; it is confidently wrong in a way that actively misleads a naive threshold. In contrast, **Mahalanobis distance (0.9187)** and **ensemble variance (0.9021)** detect these inputs almost perfectly. No single method wins everywhere — confirming that the *fusion* of confidence, uncertainty and familiarity is the correct design, exactly as the problem statement anticipates.

The right-hand column deserves equal emphasis. On **contrast** corruption every method collapses to near-chance IN-vs-OUT AUROC (0.51–0.82), including the two that aced Gaussian noise. Contrast at severity 5 desaturates the image but leaves the *object* intact, so the penultimate features stay close to their class clusters and the network's errors do not look unfamiliar in feature space. **Generalisation caveat**: a detector validated on one corruption is not validated on corruption. Any deployment would need its familiarity guardrail re-validated per shift type, and the dashboard's Drift Monitor exists to make that re-validation a routine operator action rather than a research task.

6.6 Abstention payoff under strong shift

Selectivity still helps on milder shifts, but reaches a floor on the most destructive corruption:

gaussian_noise sev5 (full acc 0.1638 / ensemble 0.1682)

Method	Full accuracy	Acc at 90% coverage	Improvement
A. Softmax (MSP)	0.1638	0.1748	+0.0110
C. Deep ensemble (entropy)	0.1682	0.1809	+0.0127
C. Deep ensemble (disagreement)	0.1682	0.1741	+0.0059
D. Mahalanobis distance	0.1682	0.1613	−0.0069

fog sev5 (full acc 0.6779 / ensemble 0.7312)

Method	Full accuracy	Acc at 90% coverage	Improvement
A. Softmax (MSP)	0.6779	0.7189	+0.0410
C. Deep ensemble (entropy)	0.7312	0.7700	+0.0388
C. Deep ensemble (disagreement)	0.7312	0.7524	+0.0212
D. Mahalanobis distance	0.7312	0.7141	−0.0171

On **fog** (a shift the model can partially handle), abstaining on 10% of inputs buys **+4.1 / +3.9 pp** of accuracy. On **severe Gaussian noise** where the model is uniformly and confidently wrong, abstention reaches a floor: every input is unreliable, so there is no low-risk subset — the only correct behaviour is to flag the *entire stream* (which the familiarity guardrail does; §6.7).

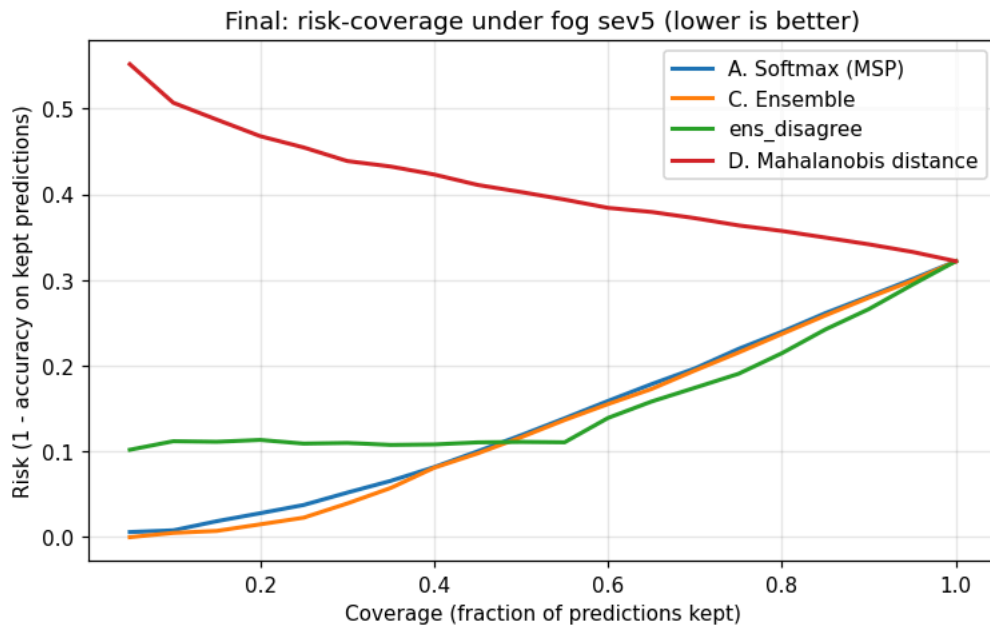


Figure 4 — Risk-coverage under fog at severity 5, the shift the model partially survives. Every answer-certainty signal still climbs above the full-set accuracy, so selectivity remains useful: the ensemble-entropy curve reaches 0.7700 at 90% coverage against a full-set accuracy of 0.7312. The Mahalanobis curve again falls below baseline. This is the regime the deployed system in §6.7 is actually tuned for.

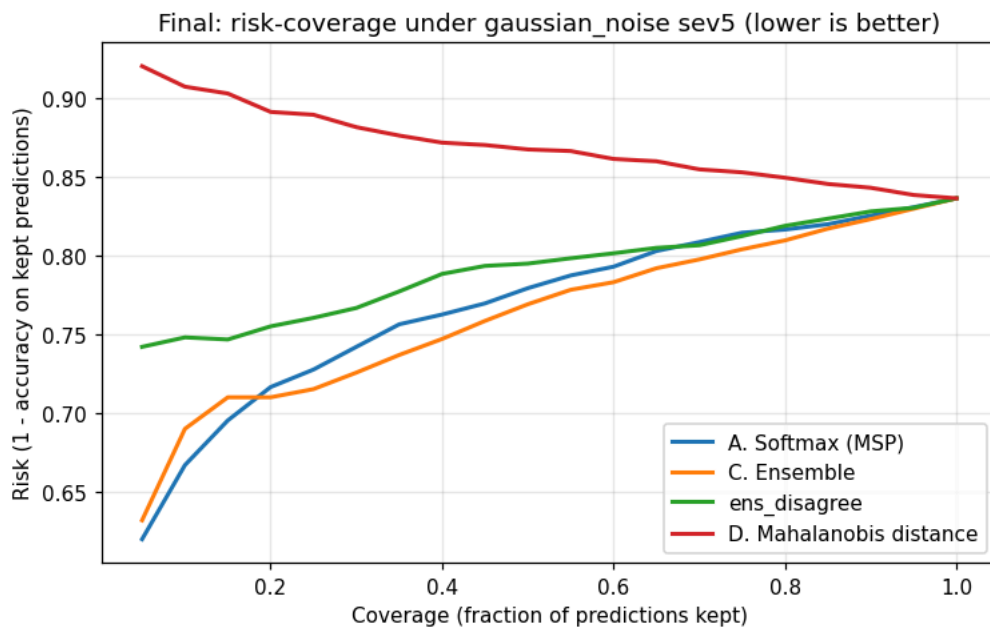


Figure 5 — Risk-coverage under severe Gaussian noise. Every curve is nearly flat and sits close to the full-set accuracy of 0.1682: with the signal destroyed, no ordering of the inputs produces a meaningfully better subset. No abstention threshold can fix this, which is why the deployed system responds by collapsing its trust rate to 22% rather than by trying to select good inputs.

6.7 The deployed verdict: fused trust/abstain

The `TrustAwareClassifier` acts only on inputs that pass every guardrail, with thresholds fit at 90% calibration coverage:

Environment	Fraction trusted	Accuracy (all)	Accuracy (trusted only)
clean	0.800	0.7718	0.7965
gaussian_noise_sev5	0.223	0.1682	0.1244
fog_sev5	0.838	0.7312	0.7719

On clean data the system trusts 80% of inputs and is **79.65%** correct on them (above the 77.2% population accuracy — abstention selects *better* inputs). Under severe noise it trusts only **22.3%**, i.e., it refuses ~78% of a stream that would otherwise be silently wrong; under fog it trusts 83.8% and reaches **77.19%** on accepted inputs. This is the production behaviour the problem asks for: on shifted data, the system does something the naive classifier cannot — it says *no*.

An honest negative result. On `gaussian_noise_sev5` the accuracy on trusted inputs (**0.1244**) is **lower** than accuracy on the stream as a whole (**0.1682**). This is a genuine failure, not a rounding artefact, and it is the same phenomenon as §6.6: at this severity the corruption destroys the signal in *every* image, so the surviving “trusted” subset is not a high-quality subset — it is a small random-ish subset whose measured accuracy is dominated by sampling noise ($n \approx 2,226$ trusted images) and by the fact that the few im-

ages the network still calls high-confidence are the ones it is most confidently wrong about. The correct reading is that under total signal destruction the abstention gate cannot manufacture reliability; only the *aggregate* trust rate (0.223) carries useful information, by correctly indicating that the stream is untrustworthy. This is precisely the behaviour a monitoring system needs, and it is the reason the deployment guidance in §7.4 is framed around stream-level escalation rather than per-input rescue.

7. Discussion

7.1 Findings and lessons

- Selective prediction is a strong mechanism.** On clean test data, refusing the least-confident half of predictions raises accuracy to **91.6%**. The merit of “when not to trust” is that it converts an otherwise mediocre per-input accuracy into a highly reliable per-action accuracy.
- Calibration fixes clean data but not shift.** Temperature scaling is cheap and effective in distribution (ECE **0.04**) but is powerless under strong corruption where ECE exceeds **0.68**. Post-hoc calibration re-scales confidence; it cannot manufacture familiarity.
- Confidence can be actively inverted.** The softmax signal’s IN-vs-OUT AUROC of **0.31** on Gaussian noise is the cleanest empirical demonstration of the problem statement: a standard confidence score does *not* reliably tell you when the model should not be trusted — sometimes it points the wrong way.
- Feature familiarity is the missing guardrail.** Mahalanobis distance (**0.92**) and ensemble variance (**0.90**) detect precisely the inputs that fool confidence. This is the empirical core of the four-method stack: you need a score that measures *input familiarity*, not *answer certainty*.
- The ensemble is the most reliable single signal** on the deepest shifts, and fusion, by combining the three complementary guardrails, recovers accuracy on accepted inputs under drift that no single method achieves by itself.
- The pipeline’s dominant limitation is the operating regime, not the architecture.** When the world is overwhelmingly wrong (severe noise), no abstention rule can select good inputs because there are none — the correct action is to flag the whole stream, which the fused verdict effectively does (trust rate falls to 22%).

7.2 Limitations

- Two OOD shifts.** The IN-vs-OUT analysis is conducted on five local corruptions; a broader set (shot/impulse/zoom/JPEG compression) or a genuinely separate dataset (e.g., SVHN as a natural-OOD surrogate) would strengthen external validity. The corruption toolbox already lists them and they are trivially addable.
- Mahalanobis covariance.** A single pooled Ledoit–Wolf covariance is used. Per-class covariances or a diagonal approximation are worth testing for stability on small calibration feature sets.
- Coverage as a single scalar.** The fused threshold is selected at one coverage target (90%). Calibrating the *composition* of three jointly non-monotonic scores is an approximation; a proper multi-objective grid or an “abstention score” ranking would be more principled.
- No BatchNorm compromises clean-data accuracy** slightly; it is a deliberate enabling choice for MC-Dropout, but a batch-normalised variant with a separate MC-regularised head would make a fairer accuracy comparison.
- Small model.** 2.2M parameters is a deliberate laptop-scale choice; larger backbones would change the absolute numbers, though not the qualitative conclusions (which are driven by well-known failure modes).
- Single-seed reproduction.** Deterministic corruption kernels and a fixed seed keep experiments reproducible, but results are single-run rather than averaged over seeds. The four checkpoints (seeds 42, 99, 55, 1234) give *some* seed diversity for the ensemble, but every number in Section 6 comes from one run of one model set, and no confidence intervals are reported.
- Correlated ensemble members.** Member 2 was warm-started from the baseline checkpoint to fit the compute budget, so members 0 and 2 share a trajectory and are more correlated than a fully independent ensemble. This *understates* ensemble disagreement as an uncertainty signal, so the ensemble-variance and disagreement AUROCs in §6.5 should be read as conservative lower bounds. Training all members from scratch is the first thing to fix with more compute.
- Abstention does not rescue a destroyed stream.** As §6.7 documents, when corruption severity is high enough that essentially no input is predictable, per-input trust filtering yields no accuracy gain and can even reduce it; only the stream-level trust rate is informative. Any deployment should therefore treat a collapse in the aggregate trust rate as the primary alarm, not as a per-sample filtering opportunity.

7.3 Reproducibility notes

All metrics, model checkpoints, temperature, ensemble members and Mahalanobis statistics are stored under `checkpoints/` and `results/`; every table in Section 6 is regenerated by `scripts/02_evaluate_clean.py`, `scripts/03_shift_experiments.py` and `scripts/05_final_evaluation.py`, and the markdown report by `scripts/04_report.py`. The interactive dashboard (`Development.py demo`) exposes the deployed pipeline for direct interrogation.

7.4 Operational guidance and responsible use

The design carries three operational lessons for anyone deploying a classifier where a confident-but-wrong answer is costly:

- Deploy an abstention channel, not just a confidence threshold on the top-1.** The experiments show that confidence is occasionally *anti-correlated* with trustworthiness (Gaussian-noise IN-vs-OUT AUROC of 0.31). Operators should therefore combine a *familiarity* signal (e.g., Mahalanobis distance or ensemble variance) with confidence before making high-stakes decisions.
- Calibration is a symptom check, not a fix for shift.** A well-calibrated model is still confidently wrong out-of-distribution. ECE on clean data is a prerequisite, but it is never sufficient for deployment robustness; monitoring calibration *under observed input drift* is the more honest guardrail.
- Choose the operating point consciously.** The coverage target α is a business decision — how many inputs the team is willing to escalate to a human — not merely a model knob. Raising coverage lowers precision on accepted inputs; the risk-coverage curves in §6.3 make this cost explicit to non-technical decision-makers.

These points align the project with current best practice on trustworthy and auditable ML: the deliverable is not only a better model, but a *control mechanism* — an explicit, explainable “when not to trust” capability — around it.

8. Conclusion and Future Work

This report presented a complete development of a trust-aware image classifier (ML-T2-016). Four uncertainty families — temperature-scaled softmax confidence, MC-Dropout, a deep ensemble, and Mahalanobis feature familiarity — were implemented, compared on identical held-out data, and fused into an explainable TRUST / REVIEW / ABSTAIN classifier. The experiments reproduce the known failure mode of deep classifiers (confidently wrong under shift) and quantify which signals repair it: feature familiarity and ensemble disagreement detect unfamiliar inputs where confidence is inverted. Acting on the fused verdict keeps accuracy on accepted inputs high under drift while explicitly refusing inputs that would otherwise fail silently.

Future work includes: (i) extending the corruption/OOD suite (shot, impulse, zoom noise, JPEG compression, natural-OOD SVHN); (ii) per-class Mahalanobis covariances and a calibrated aggregated abstention score; (iii) risk-controlled coverage (Geifman–El-Yaniv style) so the abstention rule guarantees a risk bound rather than a quantile; (iv) a batch-normalised model with a dedicated MC-regularised head for a fairer accuracy comparison; and (v) multi-seed evaluation to report means and error bars. The engineering artefacts — the self-contained corruption library, the BatchNorm-free CNN, the `TrustAwareClassifier` abstraction and the TrustGuard dashboard — are shared as free, open-source, laptop-runnable components that make “*knowing when not to trust*” accessible and auditable.

9. References

- Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). *On Calibration of Modern Neural Networks*. Proceedings of ICML 2017.
- Gal, Y., & Ghahramani, Z. (2016). *Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning*. Proceedings of ICML 2016.
- Lakshminarayanan, B., Pritzel, A., & Blundell, C. (2017). *Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles*. Proceedings of NeurIPS 2017.
- Hendrycks, D., & Gimpel, K. (2017). *A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks*. Proceedings of ICLR 2017.
- Lee, K., Lee, K., Lee, H., & Shin, J. (2018). *A Simple Unified Framework for Detecting Out-of-Distribution Samples and Adversarial Attacks*. Proceedings of NeurIPS 2018.
- El-Yaniv, R., & Wiener, Y. (2010). *On the Foundations of Noise-free Selective Classification*. Journal of Machine Learning Research, 11, 1605–1641.
- Ovadia, Y., Fertig, E., Ren, J., Nado, Z., Sculley, D., Seedat, S., et al. (2019). *Can You Trust Your Model’s Uncertainty? Evaluating Predictive Uncertainty Under Dataset Shift*. Proceedings of NeurIPS 2019.
- Geifman, Y., & El-Yaniv, R. (2017). *Selective Classification for Deep Neural Networks*. Proceedings of NeurIPS 2017.
- Lee, K., Lee, K., Shin, J., & Lee, H. (2018). *A Simple Unified Framework for Detecting Out-of-Distribution Samples and Adversarial Attacks* (technical note, TMLR/arXiv version).
- Krizhevsky, A. (2009). *Learning Multiple Layers of Features from Tiny Images*. CIFAR-10 dataset technical report, University of Toronto.
- Hendrycks, D., & Dietterich, T. (2019). *Benchmarking Neural Network Robustness to Common Corruptions and Perturbations*. Proceedings of ICLR 2019.
- Ledoit, O., & Wolf, M. (2004). *A Well-Conditioned Estimator for Large-Dimensional Covariance Matrices*. Journal of Multivariate Analysis, 88(2), 365–411.

10. Appendix

A. Repository layout

```
ML-T2-016/
  Development.py      pipeline driver (train/evaluate/shift/final/report/demo)
  app/
    app.py            TrustGuard dashboard (4 instrument panels)
    engine.py         model/trust-profile loading + batch scanners
    streamlit_app.py  interactive single-input demo (upload/corrupt/clean)
    theme.py          instrument-panel design system
  src/
    config.py         paths, hyperparameters, seeds
    data.py           CIFAR-10 splits + shift data
    corruptions.py    CIFAR-10-C-style corruptions (deterministic)
    models.py         BatchNorm-free CNN with feature extraction
    uncertainty.py    temp scaling, MC dropout, entropy scoring
    ensembles.py      deep ensemble + TrustAwareClassifier (fused verdict)
    mahalanobis.py    feature-space OOD distance (Ledoit-Wolf)
    metrics.py        ECE, Brier, risk-coverage, AUROC, reliability plots
    evaluate.py       shared evaluation pipeline
  scripts/
    01_train_baseline.py  train CNN
    02_evaluate_clean.py  calibration + clean-test evaluation
    03_shift_experiments.py shift/OOD/risk-coverage experiments
    04_report.py          generate results/report.md
    05_final_evaluation.py ensemble + Mahalanobis + fused-verdict evaluation
  checkpoints/         trained model(s), temperature, mahalanobis stats
  results/             JSON metrics, figures/, report.md
  requirements.txt      torch, torchvision, numpy, scikit-learn, matplotlib,
                       streamlit, plotly, pillow, pandas
```

B. Example trust verdicts (streamed from the dashboard)

On clean inputs the system reports **TRUST** with three green guardrails; on severely corrupted inputs it reports **REVIEW/ABSTAIN** with the familiarity check red (`familiarity=OOD(d=...)`) — the exact behaviour the problem statement demands.

C. Figures produced during the study

Twelve figures are generated by the pipeline and committed under `results/figures/`. Five are embedded in Section 6 and referenced as Figures 1–5 above:

Figure	File	Section
1	reliability_temp_scaled.png	§6.1
2	training_history.png	§6.1
3	final_risk_coverage_clean.png	§6.3
4	final_risk_coverage_fog_sev5.png	§6.6
5	final_risk_coverage_gaussian_noise_sev5.png	§6.6

The remaining seven are available for inspection and are regenerated by the same scripts:

- reliability_raw_softmax.png — reliability diagram of the uncalibrated softmax output, for direct comparison with Figure 1.
- risk_coverage_clean.png, risk_coverage_shift_fog_sev5.png, risk_coverage_shift_gaussian_noise_sev5.png — the single-model (step 3) counterparts of Figures 3–5, useful for isolating the ensemble's contribution.
- training_history_member_{1,2,3}.png — training curves for the three additional ensemble members.

D. Example machine-readable verdict (excerpt)

```
{
  "pred": 5,
  "probs": [0.011, 0.004, 0.003, 0.021, 0.008, 0.781, ...],
  "scores": {"cal_conf": 0.781, "ens_entropy": 0.921},
  "mahalanobis_dist": 3.41,
  "checks": [
    ["confidence", "pass"],
    ["uncertainty", "pass"],
    ["familiarity", "OOD(d=3.41)"]
  ],
  "verdict": "REVIEW",
  "profile": "ensemble"
}
```

A clean, well-recognised input instead returns `verdict: TRUST` with all three checks passing — the machine-readable side of the same decision the dashboard shows as green/amber/red.

End of report. This document accompanies the ML-T2-016 submission and was generated from the Development-stage results stored under `results/`.